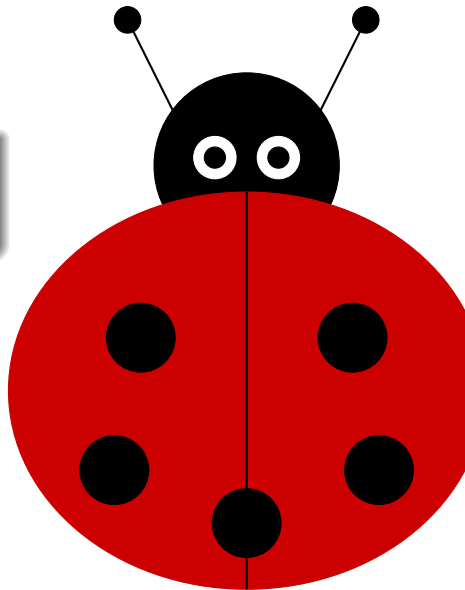


Java Bug Hunt

Find It · Fix It · Prevent It

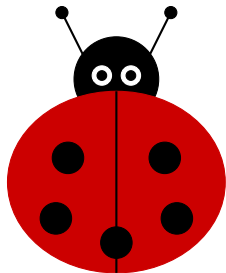
Hands-On Java Debugging Workshop

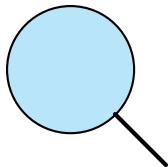
4-Hour Workshop



"What's the nastiest bug you've ever shipped?"

- 2–3 volunteers share, 60 seconds each
- Goal: normalize bugs as part of the job, not a mark of failure
- Everyone in this room has shipped one — today we build the muscle to catch it earlier





Don't ask:

"Where is the bug?"

Ask:

"What evidence tells me where the bug is?"

This reframe turns the workshop from a list of Java gotchas into a real, repeatable debugging *skill*.

Workshop Overview

- **Duration:** 4 hours
- **Audience:** Beginner to intermediate Java programmers
- **Format:** Interactive, hands-on, VS Code throughout
- **Split:** ~25% explanation · ~65% hands-on · ~10% final challenge
- **Group size:** Works for a classroom up to ~30, in pairs

What You'll Need

- Laptop with a JDK (17 or 21 LTS) and VS Code installed *before* the session
- Extension Pack for Java (Microsoft) installed
- A notebook or doc for tracking bugs found (symptom / root cause / fix)
- Curiosity, not confidence — you are expected to be wrong before you're right today

Learning Objectives

By the end, participants will be able to:

- 1 Tell apart compile-time, runtime, and logical bugs
- 2 Read a stack trace and trace an exception to its root cause
- 3 Use the VS Code Java debugger instead of guessing
- 4 Explain and avoid common Java traps (`null`, `==` vs `.equals()`, immutability, hashing)
- 5 Diagnose bugs in collections, OOP, and exception handling
- 6 Apply a systematic debugging process to unfamiliar code

Setup Guide: Installing Java in VS Code

Send this out the day before the workshop.

- ① Install a JDK (17 or 21 LTS) — Eclipse Temurin or Oracle JDK
- ② Install VS Code
- ③ Install the **Extension Pack for Java** (Microsoft)
- ④ Verify with a "Hello World" run *and* a breakpoint test

Step 1: Install a JDK

```
1  # In a terminal, verify installation:  
2  java -version  
3  javac -version
```

- Both commands should print a version (e.g. 17.0.x)
- command not found? The JDK's bin folder isn't on your PATH
- Windows: fix via Environment Variables macOS/Linux: fix via ~/.zshrc or ~/.bashrc

Step 2 & 3: VS Code + Extensions

Install VS Code: <https://code.visualstudio.com>

Extensions view (Ctrl+Shift+X) → search "**Extension Pack for Java**"

Bundles:

- Language Support for Java (IntelliSense, live error checking)
- **Debugger for Java** ← the one we use most
- Test Runner for Java (JUnit support)
- Maven for Java / Project Manager for Java
- IntelliCode (optional, AI-assisted completions)

Step 4: Verify the Debugger Works

```
1 public class Bug {  
2     public static void main(String[] args) {  
3         System.out.println("Setup works!");  
4     }  
5 }
```

- 1 Click **Run** above main, or press F5
- 2 Set a breakpoint (click the gutter) on the `println` line
- 3 Press F5 again → execution should pause, Variables panel appears

If that panel shows up, you're fully ready for today.

Common Setup Issues

Symptom	Likely Fix
java: command not found	JDK not on PATH
No "Run" button above main	Extension Pack missing / wrong folder root
Wrong JDK used	Java: Configure Java Runtime
Debugger ignores breakpoints	Reload VS Code, check extension enabled

Overall Workshop Flow (Part 1)

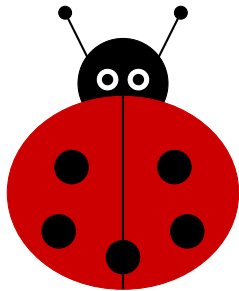
Time	Session	
0:00–0:15	Mission 1: What Is a Bug?	
0:15–0:45	Mission 2: Read the Clues	
0:45–1:20	Mission 3: Objects & References	
1:20–1:50	Mission 4: Java's Sneaky Traps	
1:50–2:00	Break	

Overall Workshop Flow (Part 2)

Time	Session	
2:00–2:35	Mission 5: Collection Crimes	
2:35–3:10	Mission 6: OOP Gone Wrong	
3:10–3:35	Mission 7: Exception Investigation	
3:35–3:55	Mission 8: Logic Bug Hunt	
3:55–4:00	Final Mission: Rapid-Fire Challenge	

The Debugging Method We'll Practice All Day

- ① **Reproduce** the bug reliably before touching code
- ② **Read** the whole stack trace, not just the top line
- ③ **Isolate** using breakpoints and variable inspection, not scattered `println`
- ④ **Hypothesize** what's wrong, in one sentence
- ⑤ **Test** the smallest possible change to confirm or deny it
- ⑥ **Fix** the root cause, not the symptom
- ⑦ **Verify** with a test that would have caught it originally



Mission 1

What Is a Bug?

0:00 – 0:15

Mission 1 — What Is a Bug? (0:00–0:15)

Predict the output before running:

```
1  int x = 10;  
2  int y = 20;  
3  System.out.println("Average = " + x + y / 2);
```

Mission 1 — What Is a Bug? (0:00–0:15)

Predict the output before running:

```
1  int x = 10;  
2  int y = 20;  
3  System.out.println("Average = " + x + y / 2);
```

Average = 1020

Mission 1 — What Is a Bug? (0:00–0:15)

Predict the output before running:

```
1  int x = 10;  
2  int y = 20;  
3  System.out.println("Average = " + x + y / 2);
```

Average = 1020

"The program compiled. So why is it wrong?"

Why Did That Happen?

- `+` on a `String` concatenates left to right
- `"Average = " + x` becomes the string `"Average = 10"`
- Then `+ y / 2` evaluates `y / 2` first (10), then concatenates it
- Result: `"Average = 10" + "10" → "Average = 1020"`

Lesson: operator precedence and type coercion combine in ways that are easy to misread.

Three Types of Bugs

Type	Example	Symptom
Compile-time	<code>int x = "Hello";</code>	Won't build
Runtime	<code>int x = 10 / 0;</code>	Builds, crashes
Logical	<code>x + y / 2</code>	Builds, runs, <i>wrong</i>

A program compiling does NOT mean it's correct.

The Fix

```
1  int average = (x + y) / 2;  
2  System.out.println("Average = " + average);
```

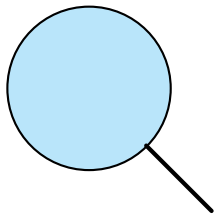
- Compute the numeric result *first*, in its own variable
- Only convert to a String at the very end, once
- **Prevention habit:** never mix arithmetic and string concatenation on one line

Mission 1: Hands-On

Mini Activity (5–7 min)

- 5 tiny code snippets
- Classify each: compilation / runtime / logical error
- Predict first — then run and verify

Debrief question: which type was hardest to predict, and why?



Mission 2

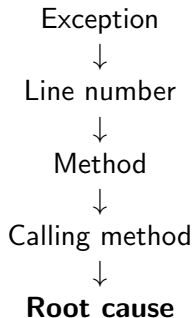
Read the Clues

0:15 – 0:45

Mission 2 — Read the Clues (0:15–0:45)

```
1 public static void main(String[] args) {  
2     process();  
3 }  
4 static void process() {  
5     calculate();  
6 }  
7 static void calculate() {  
8     String name = null;  
9     System.out.println(name.length());  
10 }
```

Reading a Stack Trace



- An error message is a *clue*, not the final answer
- Click a stack trace line in VS Code's Debug Console to jump to it
- Read *top to bottom*: the top frame is where it broke, lower frames are how you got there

Symptom vs. Root Cause

- **Symptom:** where the exception was thrown (`calculate()`, line 9)
- **Root cause:** why `name` was `null` in the first place
- Fixing the symptom (a null check right before `.length()`) can hide the real bug
- Fixing the root cause means asking: *should `name` ever be allowed to be null here?*

Print vs. Debugger

	<code>println</code> debugging	Debugger (breakpoints)
Setup	None	Click gutter, run in Debug mode
State visibility	Only what you printed	Every variable in scope, live
Iteration cost	Edit, save, rerun each time	Pause, inspect, step, no edits
Best for	Very simple, one-off checks	Anything non-trivial

Today, default to the debugger — that's the muscle we're building.

Mission 2: Hands-On — "Read the Crime Scene" (20 min)

3 programs, each with a different exception:

- `NullPointerException`
- `ArrayIndexOutOfBoundsException`
- `ArithmeticException`

For each, answer: **What? Where? Why? Fix?** *Use the VS Code debugger, not just reading*
— *set a breakpoint before the crash line and inspect variables.*



Mission 3

Objects & References

0:45 – 1:20

Mission 3 — Objects & References (0:45–1:20)

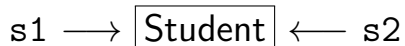
Bug #1 — null

```
1 Student s = null;  
2 System.out.println(s.getName());
```

Bug #2 — Two references, one object

```
1 Student s1 = new Student("Arup");  
2 Student s2 = s1;  
3 s2.setName("Rahul");  
4 System.out.println(s1.getName());
```

Who Changed s1?



Both variables point at the **same** object — there's only one `Student`.

- Bug #3: Assignment copies the *reference*, not the object
- Bug #4: Shallow vs. deep copy (nested Address field)

Bug #5 — Arrays Are References Too

```
1 static void zeroOut(int[] arr) {  
2     arr[0] = 0;  
3 }  
4  
5 int[] numbers = {1, 2, 3};  
6 zeroOut(numbers);  
7 System.out.println(numbers[0]);    // 0, not 1!
```

- Java passes the *reference* to the array into the method
- The method mutates the same array the caller sees
- Contrast with primitives: passing an `int` passes a *copy*

The Fix: Defensive Copying

```
1 Student copy = new Student(original.getName(),  
2                      new Address(original.getAddress()));
```

- Deep-copy nested mutable fields when you need true independence
- Or make the class **immutable** so this class of bug can't happen at all
- **Prevention habit:** ask "who else holds a reference to this?" before mutating an object

Mission 3: Hands-On (20–25 min)

- Modifying one `Student` unexpectedly changes a second
- **Mission:** find out why two "different" students change together
- Use the VS Code Variables panel to inspect references side by side
- Bonus: fix it two different ways (defensive copy vs. immutability)



Mission 4

Java's Sneaky Traps

1:20 – 1:50

Mission 4 — Sneaky Traps (1:20–1:50)

Trap #1 — == vs .equals()

```
1 String a = new String("Java");  
2 String b = new String("Java");  
3 System.out.println(a == b);           // ?  
4 System.out.println(a.equals(b));      // ?
```

Trap #2 — The string pool

```
1 String a = "Java";  
2 String b = "Java";  
3 System.out.println(a == b);           // ?
```

Trap #3 — Integer Wrapper Surprise

```
1 Integer a = 100;  
2 Integer b = 100;  
3 System.out.println(a == b);    // true  
4  
5 Integer x = 200;  
6 Integer y = 200;  
7 System.out.println(x == y);    // false
```

Let them debug this themselves — the Integer cache (−128 to 127) is a great "aha" moment.

Trap #4 — String Immutability

```
1 String s = "Hello";  
2 s.concat(" Java");  
3 System.out.println(s);    // still "Hello" ?
```

"Why didn't *s* change?" → `concat()` returns a **new** String; nothing reassigned *s*.

Trap #5 — Switch Fallthrough

```
1  int day = 2;
2  switch (day) {
3      case 1:
4      case 2:
5          System.out.println("Early week");
6      case 3:
7          System.out.println("Midweek");
8          break;
9      default:
10         System.out.println("Other");
11 }
12 // prints BOTH "Early week" and "Midweek"
```

Missing break lets execution fall through to the next case. **Fix:** add break, or prefer Java's arrow-style switch expressions which don't fall through.

Trap #6 — Autoboxing NPE

```
1 Map<String, Integer> scores = new HashMap<>();  
2 int score = scores.get("missingKey");    // NullPointerException!
```

- `get()` returns `null` for a missing key
- Unboxing `null` into a primitive `int` throws `NullPointerException`
- **Fix:** use `Integer` (nullable) or `getOrDefault("missingKey", 0)`

Mission 4: Prevention Checklist



- Compare objects with `.equals()`, never `==`, unless identity is truly what you mean
- Never rely on `Integer ==` working past the `-128` to `127` cache range
- Remember Strings (and most wrapper types) are **immutable** — methods return new values
- Always terminate `switch` cases with `break` or `return`, or use arrow-style `switch`
- Be careful unboxing values that might be `null`



Break

1:50 – 2:00

Stretch, refill your coffee, and swap seats if you'd like a new debugging partner for the second half.



Mission 5

Collection Crimes

2:00 – 2:35

Mission 5 — Collection Crimes (2:00–2:35)

Crime #1 — Removing while iterating

```
1  for (String name : names) {  
2      if (name.startsWith("A")) {  
3          names.remove(name);  // ConcurrentModificationException  
4      }  
5  }
```

Fix: explicit `Iterator.remove()` or `names.removeIf(...)`

Crime #2 — Mutable Keys in a HashMap

```
1 Map<Student, Integer> marks = new HashMap<>();  
2 marks.put(student, 90);  
3  
4 student.setId(200);    // id used in hashCode()  
5 marks.get(student);    // returns null!
```

"Why can't the HashMap find something that is clearly there?"

- The bucket was computed from the *old* hash code at insertion time
- **Fix:** never mutate fields used in `hashCode()` after inserting into a hash-based collection

Crime #3 & #4

Crime #3 — equals() without hashCode()

```
1 Set<Student> students = new HashSet<>();  
2 students.add(new Student(101));  
3 students.add(new Student(101));  
4 System.out.println(students.size());    // 2, not 1!
```

Crime #4 — Raw collections

```
1 List list = new ArrayList();  
2 list.add("Java"); list.add(100);  
3 String s = (String) list.get(1);    // ClassCastException
```

Crime #5 — Arrays.asList Is Fixed-Size

```
1 List<String> names = Arrays.asList("Alice", "Bob");  
2 names.add("Carol");    // UnsupportedOperationException
```

- Arrays.asList returns a fixed-size view backed by the array, not a resizable ArrayList
- **Fix:** wrap it — new ArrayList<>(Arrays.asList(...))

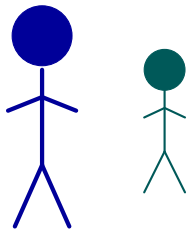
Mission 5: Prevention Checklist



- Never mutate a collection while iterating it directly — use an `Iterator` or `removeIf`
- Never mutate a field used in `hashCode()`/`equals()` once the object is in a hash-based collection
- Always override `hashCode()` whenever you override `equals()`
- Prefer generic collections (`List<String>`) over raw types — catch mismatches at compile time
- Know which factory methods return fixed-size or immutable collections before you mutate them

Mission 5: Hands-On (15–20 min)

- Debug a "Student Roster" program combining 2–3 crimes
- Use breakpoints to inspect `hashCode()/equals()` behavior live
- Bonus: rewrite one loop to use `removeIf` instead of manual iteration



Mission 6

OOP Gone Wrong

2:35 – 3:10

Mission 6 — OOP Gone Wrong (2:35–3:10)

Bug #1 — Overloading vs. overriding

```
1  class Parent { void show() { System.out.println("Parent"); } }
2  class Child extends Parent { void show() { System.out.println("Child"); }
   }
3
4  Parent p = new Child();
5  p.show();    // which one runs?
```

Dynamic dispatch: the runtime type decides.

Bug #2 — Downcasting

```
1 Parent p = new Parent();  
2 Child c = (Child) p;    // compiles... then ClassCastException
```

Compile-time type $\neq$ Runtime type

Fix: check first with `instanceof` (or a pattern-matched `instanceof`) before casting.

Bug #3 — Overridable Calls in Constructors

```
1  class Parent {  
2      Parent() { show(); }  
3      void show() { System.out.println("Parent"); }  
4  }  
5  class Child extends Parent {  
6      int level = 5;  
7      void show() { System.out.println("Level: " + level); }  
8  }
```

Constructing a `Child` calls `Parent()`, which calls the **overridden** `show()` — but `Child`'s fields aren't initialized yet, so `level` prints as 0, not 5.

Bug #4 — super vs this

```
1  class Parent {  
2      String label = "parent";  
3  }  
4  class Child extends Parent {  
5      String label = "child";  
6      void printBoth() {  
7          System.out.println(this.label);    // "child"  
8          System.out.println(super.label);    // "parent"  
9      }  
10 }
```

Fields are **shadowed**, not overridden — both copies exist. Debug with breakpoints in both constructors to see each field populate.

Bug #5 — Broken equals() Symmetry with Inheritance

```
1  class Point { int x, y; /* equals() checks getClass() == getClass() */ }
2  class ColorPoint extends Point { String color; }
3
4  Point p = new Point(1, 1);
5  ColorPoint cp = new ColorPoint(1, 1, "red");
6  p.equals(cp);    // ?
7  cp.equals(p);    // ?
```

Comparing across a subclass boundary breaks the equals() contract (symmetry) if not handled carefully. **Rule of thumb:** favor composition over inheritance when equality matters.

Mission 6: Hands-On

- Small 2-level hierarchy program with a shadowed field and an overridable-constructor call
- Debug the unexpected value using breakpoints in both parent and child constructors
- Predict the output *before* running, then verify



Mission 7

Exception Investigation

3:10 – 3:35

Mission 7 — Exception Investigation (3:10–3:35)

Common mistakes:

- ❶ Catching everything with `catch (Exception e)`
- ❷ Swallowing exceptions (empty catch block)
- ❸ Wrong exception type caught
- ❹ Losing the original cause when rethrowing
- ❺ Not using `try-with-resources`

Checked vs. Unchecked Exceptions

	Checked	Unchecked
Examples	IOException, SQLException	NullPointerException, IllegalArgumentException
Compiler enforces handling?	Yes	No
Typical cause	External failure (file, network)	Programming mistake

Catching `Exception` broadly hides which kind you actually got — and whether it was even recoverable.

Exception Challenge (15–20 min)

```
1 try {  
2     process();  
3 } catch (Exception e) {  
4     System.out.println("Something went wrong");  
5 }
```

"How would you change this so debugging becomes easier?"

```
1 } catch (Exception e) {  
2     e.printStackTrace();  
3     // or: logger.error("process() failed", e);  
4 }
```

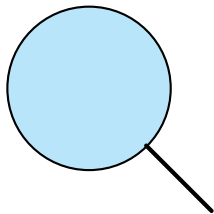
Losing the Cause When Rethrowing

```
1  // BAD: original stack trace is gone
2  catch (IOException e) {
3      throw new RuntimeException("Failed to load file");
4  }
5
6  // GOOD: original cause preserved
7  catch (IOException e) {
8      throw new RuntimeException("Failed to load file", e);
9  }
```

The second form keeps the full chain — essential when debugging in production logs.

Try-With-Resources

```
1  // BAD: leaks the stream if readLine() throws
2  BufferedReader r = new BufferedReader(new FileReader(path));
3  String line = r.readLine();
4  r.close();
5
6  // GOOD: resource always closed, even on exception
7  try (BufferedReader r = new BufferedReader(new FileReader(path))) {
8      String line = r.readLine();
9  }
```



Mission 8

Logic Bug Hunt

3:35 – 3:55

Mission 8 — Logic Bug Hunt (3:35–3:55)

```
1  // Bug #1: off-by-one  
2  for (int i = 0; i <= arr.length; i++)  
3  
4  // Bug #2: incorrect boundary  
5  if (marks > 40)    // should be >= 40  
6  
7  // Bug #3: infinite loop  
8  int i = 0;  
9  while (i < 10) { System.out.println(i); }  
10  
11 // Bug #4: operator precedence  
12 if (age > 18 && registered || admin)
```

Bug #5 — Integer Division Truncation

```
1  int total = 7;
2  int count = 2;
3  double average = total / count;    // 3.0, not 3.5!
```

- `total / count` performs **integer** division first, truncating to 3
- Only *then* does it widen to double
- **Fix:** `(double) total / count`, or make one operand a double up front

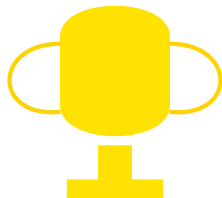
Bug #6 — Floating-Point Comparison

```
1 double price = 0.1 + 0.2;  
2 System.out.println(price == 0.3);    // false !
```

- Binary floating point can't represent 0.1 exactly
- Never compare double/float with ==
- **Fix:** compare with a tolerance ($\text{Math.abs}(a - b) < 1\text{e-}9$), or use `BigDecimal` for money

Mission 8: Hands-On (10–15 min)

- No exception here — just a silently wrong result
- Use breakpoints + Watch panel to step value-by-value
- Catch the exact iteration where behavior diverges from expectation
- This is the category no static analysis tool reliably catches — tracing is the only way



Final Mission

The Ultimate Bug Hunt

3:55 – 4:00

Final Mission — The Ultimate Bug Hunt (3:55–4:00)

Student Management System with 5–7 seeded bugs:

- `NullPointerException` · `==` vs `.equals()`
- Collection bug · Inheritance/overriding bug
- Logical error · `equals()/hashCode()` problem
- Swallowed exception

Find at least 5 bugs in 5 minutes.

Bug Report Template

Bug	Symptom	Root Cause	Fix
1	Wrong student found	== used	.equals()
2	Program crashes	Unchecked null	Add null check
3	Wrong average	Loop boundary	Correct condition

Scoring: 1 point per verified fix, +1 bonus for correctly naming the bug category.

Your Debugging Toolkit, Recap

- **Breakpoints** — pause execution exactly where you need to look
- **Step over / step into / step out** — control how deep you follow a call
- **Variables panel** — inspect live state without printing anything
- **Watch expressions** — track a specific value across many steps
- **Conditional breakpoints** — pause only when a condition is true (e.g. `i == 47`)

Master Prevention Checklist



- Compare objects with `.equals()`; compare primitives and check nulls explicitly
- Override `hashCode()` whenever you override `equals()`
- Never mutate fields used in hashing after insertion into a hash-based collection
- Prefer immutability wherever practical — fewer places for shared state to go wrong
- Use generics; avoid raw collection types
- Never swallow an exception silently — log it or propagate it with its cause
- Never compare floating-point values with `==`
- Write a test for every bug you fix, so it can't silently come back

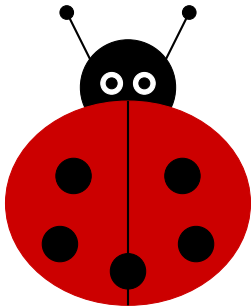
Learning Outcomes

By the end, participants can:

- 1 Distinguish compilation, runtime, and logical errors
- 2 Read and interpret stack traces
- 3 Use the VS Code Java debugger live
- 4 Identify object/reference bugs
- 5 Recognize string, wrapper, and equality traps
- 6 Debug collections and generics issues
- 7 Identify inheritance, overriding, and casting errors
- 8 Handle exceptions without swallowing them
- 9 Trace and fix logical errors in loops/conditions
- 10 Apply a systematic debugging process

Take It Further

- Turn on IDE inspections for null-safety and unused code
- Add a static analyzer (SpotBugs, PMD, or Error Prone) to your own project's build
- Practice the "predict before you run" habit on your own code, not just today's exercises
- Revisit this deck's Prevention Checklist slide monthly — habits fade without repetition



Thank You!

Questions?

Keep hunting.